

Genre Kits

Provides niche-oriented extensions and cross-cutting foundation features.

MPTK output routing: When a kit's `outputDeviceId` is empty, you can use `outputPreset` (`MidiOutputRoutingPreset`) to route commonly to an MPTK virtual output (`mptk:internal`) or similar. Bundled preset: `Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset` . For details, see [Maestro / MPTK Integration — Sample scenes / output presets / editor audition](#).

Kit / feature	Namespace	Assembly Definition	Scripting Define Symbol
Network MIDI sync	<code>jp.kshoji.unity.midi.net</code>	<code>jp.kshoji.midi.net</code>	<code>FEATURE_MIDI_NETWORK</code>
MidiClockSync	<code>jp.kshoji.unity.midi</code>	<code>jp.kshoji.midi</code> (core)	not required
Chord & scale detection	<code>jp.kshoji.unity.midi / .util</code>	<code>jp.kshoji.midi</code> (core)	not required
Input System bridge	<code>jp.kshoji.unity.midi.integrations.inputsystem</code>	<code>jp.kshoji.midi.inputsystem</code>	<code>FEATURE_INPUT_SYSTEM</code>
Foundation	<code>jp.kshoji.unity.midi.foundation</code>	<code>jp.kshoji.midi.foundation</code>	not required

Location:

Kit	Location
Networking	<code>Assets/MIDI/Scripts/Integrations/Networking/</code>
Clock / chord detection (core)	<code>Assets/MIDI/Scripts/Gameplay/</code> (incl. <code>Theory/</code>)
Input System	<code>Assets/MIDI/Scripts/Integrations/InputSystem/</code>
Foundation	<code>Assets/MIDI/Scripts/Foundation/</code>
Foundation UI	<code>Assets/MIDI/UI/Foundation/</code>
Samples	<code>Assets/MIDI/Samples/Gameplay/</code> / <code>Assets/MIDI/Samples/Integrations/Networking/</code> / <code>Assets/MIDI/Samples/Integrations/InputSystem/</code> / <code>Assets/MIDI/Samples/Foundation/</code>

Dependencies between kits

Kit / feature	Main dependency components
Network MIDI sync	<code>SmfPlayer</code> , <code>MidiManager</code>
Input System bridge	<code>MidiManager</code>
Timeline integration	<code>SmfPlayer</code> , <code>TempoMapExtractor</code> , <code>MidiRecorder</code>
Animator integration	<code>MidiInputRouter</code> , <code>MidiCcSmoother</code> , <code>MidiNoteTracker</code>

Network MIDI sync

Synchronizes MIDI events and SMF playback position over a LAN. It has a built-in lightweight UDP-based transport, so no additional packages such as Netcode / Mirror are required.

Prerequisites

Add the scripting define symbol in Project Settings:

- `FEATURE_MIDI_NETWORK`

Item	Value
Assembly Definition	<code>jp.kshoji.midi.net</code>
Settings path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Key components

Component	Role
<code>MidiNetworkHub</code>	Host. Distributes MIDI events
<code>MidiNetworkClient</code>	Injects received events into a virtual device
<code>MidiNetworkMessage</code>	Serializable DTO
<code>MidiPlaybackSync</code>	<code>SmfPlayer</code> playback-position sync (<code>broadcastInterval</code> default 0.1s, <code>seekThreshold</code> default 0.05s)
<code>MidiSessionDiscovery</code>	LAN session UDP advertisement (<code>hubPort</code> in payload)
<code>MidiSessionDiscoveryListener</code>	LAN session UDP listen (<code>OnSessionAdvertisement</code>)
<code>MidiLatencyCompensation</code>	RTT helpers (UDP Ping/Pong → <code>estimatedRttMs</code>)

Sync modes

Mode	Use
Broadcast	Distribute host input to all clients
Merge	Merge all client input on the host
Playback	Sync playback position only

Optional: Mirror / Netcode / WSNet2 bridges

UDP `MidiNetworkHub` / `MidiNetworkClient` need no extra packages. Optional bridges carry the same `MidiNetworkMessage` / `MidiNetworkMessageCodec` over a game transport you already use. Packages are **not** bundled in this repository.

Symbol	Assembly	Component	Package / gate
FEATURE_MIRROR (+ FEATURE_MIDI_NETWORK + MIRROR)	jp.kshoji.midi.net.mirror	MidiMirrorBridge	Install Mirror; MIRROR from Mirror's defines
FEATURE_NETCODE (+ FEATURE_MIDI_NETWORK + MIDI_HAS_NETCODE)	jp.kshoji.midi.net.netcode	MidiNetcodeBridge	UPM com.unity.netcode.gameobjects (versionDefines → MIDI_HAS_NETCODE)
FEATURE_WSNET2 (+ FEATURE_MIDI_NETWORK + MIDI_HAS_WSNET2)	jp.kshoji.midi.net.wsnet2	MidiWsnet2Bridge	Copy WSNet2 client + WSNet2.Runtime.asmdef (see Optional helper); Editor syncs MIDI_HAS_WSNET2 . Lobby/Game servers must run separately

Enablement (summary):

1. Add FEATURE_MIDI_NETWORK , then the bridge symbol(s) above.
2. Install the matching framework (do not leave Mirror / NGO / WSNet2 on distribution branches unless your game already depends on them).
3. Wire Host/Master hub + client/peer playbackSync as in [Networking Integration](#).
4. Open the sample scenes under Assets/MIDI/Samples/Integrations/Networking/Scenes/ .

Samples

Scene	Requires
.../Networking/Scenes/MidiNetworkJamSampleScene.unity	FEATURE_MIDI_NETWORK
.../Networking/Scenes/MidiMirrorNetworkSampleScene.unity	Mirror + FEATURE_MIRROR
.../Networking/Scenes/MidiNetcodeNetworkSampleScene.unity	NGO + FEATURE_NETCODE
.../Networking/Scenes/MidiWsnet2NetworkSampleScene.unity	WSNet2 + servers + FEATURE_WSNET2

Cross-cutting foundation

Provides external MIDI Clock sync, chord & scale detection, and an Input System bridge. See also [Gameplay Components](#).

MidiClockSync

Synchronizes to an external MIDI Clock (Timing Clock / Start / Stop / Continue) and provides BPM, beat position, and bar position.

Property / API	Description
<code>pulsesPerQuarterNote</code>	Clock pulses per quarter note (MIDI standard 24)
<code>beatsPerBar</code>	Beats per bar (for <code>onBar</code> , default 4)
<code>deviceIdFilter</code>	Empty = all devices
<code>EstimatedBpm</code>	BPM estimated from recent pulses
<code>IsBpmStable</code>	Whether the BPM estimate is stable
<code>onBeat</code> / <code>onBar</code>	Beat / bar boundary events
<code>onStarted</code> / <code>onStopped</code>	Start / Stop received events

`SmfPlayerClockAdapter` switches behavior with `SmfPlayerClockMode` :

Mode	Behavior
<code>Follow</code>	Reflect the external Clock's estimated BPM into <code>SmfPlayer.tempoBpm</code>
<code>Step</code>	Advance the SMF playback position by one beat on each external Clock beat
<code>Free</code>	Ignore the external Clock

Limitations: Song Position Pointer (SPP), MTC, and Ableton Link are not supported.

Chord & scale detection

Component / utility	Role
<code>ChordRecognition</code>	Infers a chord name (<code>C</code> , <code>Cm7</code> , etc.) from the set of held notes
<code>ScaleUtility</code>	Scale-membership detection extending <code>MidiScaleUtility</code>
<code>MidiChordDetector</code>	Works with <code>MidiNoteTracker</code> ; chord-change / out-of-scale detection / target-chord quiz

Input System bridge (optional)

Component	Role
<code>MidiSyntheticDevice</code>	Synthetic Input Device (128 notes + 128 CCs + channel-specific axes)
<code>MidiInputSystemBridge</code>	MIDI → Input System state injection. SysEx / Raw UMP via <code>onMessage</code>
<code>InputSystemToMidiBridge</code>	Input Action → MIDI send
<code>MidiInputSystemMapping</code>	ScriptableObject of Action ↔ MIDI conditions

Prerequisites: package `com.unity.inputsystem` , symbol `FEATURE_INPUT_SYSTEM` .
For details on setup and the Synthetic Device layout, see [Unity Ecosystem Integrations — Input System](#).

Create a `MidiInputSystemMapping` via `Assets > Create > MIDI > Input System > Mapping` . Main fields of `MidiActionBinding` :

Field	Description
<code>actionName</code>	Action name inside the <code>.inputactions</code>
<code>messageType</code>	NoteOn / ControlChange / PitchWheel / ProgramChange, etc.
<code>group</code>	0–15, <code>-1</code> = all groups
<code>channel</code>	0–15, <code>-1</code> = all channels
<code>controllerOrNote</code>	CC number or note number
<code>valueFilter</code>	Value filter, <code>-1</code> = all
<code>targetControlIndex</code>	CC fallback target (<code>-1</code> = <code>controllerOrNote</code>)
<code>preferDedicatedControl</code>	Prefer a dedicated axis (pitch / program / channelPressure / systemPulse)
<code>invertAxis</code>	Invert the normalized axis

Samples

Scene	Description
<code>Assets/MIDI/Samples/Gameplay/Scenes/MidiClockSyncSampleScene.unity</code>	Clock injection / BPM estimation / <code>SmfPlayerClockAdapter</code>
<code>Assets/MIDI/Samples/Gameplay/Scenes/ChordScaleSampleScene.unity</code>	Chord recognition and scale quiz
<code>Assets/MIDI/Samples/Gameplay/Scenes/ChordPuzzleSampleScene.unity</code>	Target-chord quiz (<code>MidiChordDetector</code>)
<code>Assets/MIDI/Samples/Gameplay/Scenes/ScalePracticeSampleScene.unity</code>	Scale practice
<code>Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity</code>	Synthetic Device demo (<code>FEATURE_INPUT_SYSTEM</code> required)
<code>Assets/MIDI/Samples/Foundation/Scenes/FoundationSampleScene.unity</code>	Device selection / latency calibration / Foundation UI shell

Related docs

- [Samples](#) — list of kit sample scenes
- [Build PostProcessing — Network Optional Kit](#)
- [Build PostProcessing — Cross-Cutting Foundation Optional Kit](#)
- [SMF Tools](#) — `SmfPlayer` / `TempoMapExtractor`
- [Gameplay Components](#) — `MidiClockSync` / `MidiCcSmoother` / chord detection